MTP Report: PDDL-Based Formation Planning for Multi-Drone Systems

Oct 23, 2026 · @naga

Title page

PDDL-BASED MISSION AND FORMATION PLANNING FOR A MULTI-DRONE SWARM SIMULATOR

A Project Report submitted by **VADITHE SAI KUMAR NAIK** ([Roll number])

in partial fulfilment of the requirements for the award of the degree of **MASTER OF TECHNOLOGY**

[Department], Indian Institute of Technology Madras, Chennai 600036 · [Month Year]

Placeholders in square brackets (roll number, department, guide, month of submission) must be filled before printing. The layout follows the IIT Madras thesis template supplied as Reference Doc.pdf.

Thesis certificate

This is to undertake that the project report titled **PDDL-Based Mission and Formation Planning for a Multi-Drone Swarm Simulator**, submitted by me to the Indian Institute of Technology Madras for the award of Master of Technology, is a bona fide record of the work done by me under the supervision of [Guide name]. The contents of this report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Student	Guide
Vadithe Sai Kumar Naik	[Guide name], [Designation]
Chennai 600036	[Department], IIT Madras
Date: [dd Month yyyy]	Signature:

Acknowledgements

I thank my guide, [Guide name], for the direction and review given throughout this project. I thank the members of the drone simulator team, whose flight-control, GUI and communication modules made it possible to test the planning layer end to end. I also thank the developers of the ENHSP planner, which is used for all plans in this work, and my family and friends for their support.

Abstract

Keywords: PDDL 2.1; automated planning; numeric planning; ENHSP; UAV swarm; formation flight; mission planning; drone simulator.

This project builds the automated-planning layer of a multi-drone swarm simulator using the Planning Domain Definition Language (PDDL). Ten missions are modelled as PDDL domain–problem pairs: point-to-point travel, forest search, V, 2-D grid, 3-D cube, parabola and figure-eight formations, a star and circle show, a planetary-orbit show and road inspection. Together the ten domains define 142 actions over typed objects, numeric fluents for battery, altitude and energy, and universal and conditional effects that move a whole formation in one step.

A Python planning service retargets each template onto the operator's map inputs, expands it to the selected fleet size and solves it with the Expressive Numeric Heuristic Search Planner (ENHSP). The resulting action sequence is converted into per-drone routes and timed tracks that the simulator executes over MAVLink-style commands. In tests run for this report, all 15 generated mission instances (1 to 9 drones) were solved, with 5 to 73 plan steps and 0.4 to 2.4 s end-to-end planning time. Nine of ten stored templates solve directly; the search template is a seed that becomes solvable only after its search lanes are generated.

Abbreviations

Term	Meaning
AIBR	Additive Interval Based Relaxation (ENHSP pre-processing)
ENHSP	Expressive Numeric Heuristic Search Planner
GBFS	Greedy Best-First Search
GCS	Ground Control Station

Term	Meaning
GUI	Graphical User Interface
MAVLink	Micro Air Vehicle Link protocol
NFZ	No-fly zone (restricted area)
PDDL	Planning Domain Definition Language
SITL	Software in the Loop
STRIPS	Stanford Research Institute Problem Solver
SYSID	Drone system identifier
UAV	Unmanned Aerial Vehicle
UDP	User Datagram Protocol
WA*	Weighted A* search

Notation

Symbol	Meaning
N	Number of drones in the swarm
B(d)	Battery fluent of drone d (model units, 0–100)
E(a, b)	Energy required to fly the edge from location a to location b
R, C, L	Grid rows, columns and cube layers
r	Ranks per wing in the V formation (N = 1 + 2r)
s0, G	Initial state and goal condition of a planning problem
π	A plan: an ordered sequence of ground actions
|A|	Number of ground actions after grounding

Chapter 1 · Introduction

A drone swarm needs more than a route: each drone must also have enough battery, a working GPS and radio, and the right place in its formation before it may move. This project expresses those rules in PDDL so that a general-purpose planner, not hand-written mission code, decides the order of actions for ten different swarm missions.

1.1 Motivation

Missions such as area search, formation transit and drone light shows are usually scripted by hand. Every new mission, fleet size or safety rule then means new control code. Automated planning separates *what is allowed* (the domain model) from *what is wanted* (the goal), so the same planner can solve any mission whose rules are written down. A plan is also a readable, checkable artefact: every step can be traced back to a precondition in the model.

1.2 Problem statement

Given a fleet of N drones, operator map inputs (source, destination, search polygon, show centre, road endpoints) and optional restricted areas, produce an ordered, executable sequence of actions that:

1. respects per-drone resource and health conditions (battery, GPS, communication, health);
2. forms and holds the requested formation, moving the whole formation only when every member is ready;
3. reaches the mission goal (destination, coverage, laps flown, landed);
4. can be converted into per-drone flight routes for the simulator.

1.3 Objectives

- Design PDDL domains for ten swarm missions using typed objects, numeric fluents and quantified conditions.
- Build problem templates that the simulator can retarget onto any map location and expand to any valid fleet size.
- Integrate the ENHSP numeric planner into the simulator through a planning service.
- Convert symbolic plans into geometric routes and timed tracks for execution.
- Measure planning performance on generated instances and record the model’s limits.

1.4 Contributions

#	Contribution	Where
1	Ten PDDL domain–problem pairs, 142 actions in total	Chapter 5, plans/
2	Formation-level actions using universal preconditions and conditional effects, so one action moves all members	Chapter 4
3	Layered-launch gate for the 3-D cube and lap counters for figure-eight, star and planet shows	Chapter 5
4	Explicit drone-to-drone link model for the parabola formation	Chapter 5
5	Template retargeting and fleet expansion integrated with ENHSP	Chapter 6
6	Benchmark of 10 templates and 15 generated instances	Chapter 7

The planning models, problem generation and planner integration are the scope of this report. Flight control, GUI and network transport belong to the wider simulator and appear only where they explain how a plan is executed.

1.5 Organisation of the report

Chapter 2 reviews automated planning and PDDL. Chapter 3 places the planning layer inside the simulator. Chapter 4 explains the modelling method common to all domains. Chapter 5 describes each of the ten missions. Chapter 6 covers plan generation, parsing and execution. Chapter 7 reports results, and Chapter 8 concludes with future work.

Chapter 2 · Background and literature review

Classical planning searches for a sequence of actions that turns an initial state into a goal state; PDDL is the standard language for writing those actions down, and numeric PDDL adds the quantities (battery, altitude, energy) that drone missions need.

2.1 Automated planning

A classical planning task is a tuple of facts, actions, an initial state and a goal [1, 2]. Each action has preconditions that must hold before it is applied and effects that add or delete facts. A plan is a sequence of actions whose successive application from the initial state

reaches a state that satisfies the goal.

$$\Pi = \langle F, A, s_0, G \rangle, \quad a = \langle pre(a), add(a), del(a) \rangle, \quad s' = (s \setminus del(a)) \cup add(a)$$

The STRIPS formalism [1] introduced this add/delete view of actions. Planning is PSPACE-complete in general, so practical planners rely on heuristic search: they estimate distance to the goal from a relaxed version of the problem and expand the most promising states first [3, 4].

2.2 The Planning Domain Definition Language

PDDL was introduced for the 1998 International Planning Competition to give planners a common input format [5]. A model is split into two files:

- **Domain file:** object types, predicates (true/false facts), numeric functions and parameterised actions. It is written once per mission type.
- **Problem file:** the concrete objects, the initial state and the goal. It changes with every scenario.

Version	Year	Feature added	Used in this project
PDDL 1.2 [5]	1998	STRIPS actions, typing, ADL (quantifiers, conditional effects)	Yes
PDDL 2.1 [6]	2003	Numeric fluents, metrics, durative actions	Numeric fluents only
PDDL 2.2 [7]	2004	Derived predicates, timed initial literals	No
PDDL 3.0 [8]	2005	State-trajectory constraints, preferences	No

The domains in this work declare the requirements `:strips`, `:typing`, `:numeric-fluents`, `:negative-preconditions`, `:equality`, `:disjunctive-preconditions`, `:universal-preconditions` and `:conditional-effects`. Durative actions are not used: every action is instantaneous, and timing is added later by the geometry layer.

2.3 Numeric planners and ENHSP

Numeric planning lets conditions compare quantities, as in (> (battery ?d) (energy-required ?from ?to)), and lets effects change them with increase and decrease. Metric-FF [9] extended the FF heuristic to linear numeric conditions. ENHSP, the Expressive Numeric Heuristic Search Planner [10, 11], supports a wider class of numeric conditions

together with universal quantifiers and conditional effects. It offers several heuristics (for example h-add and h-max over an interval-based relaxation) and search strategies (WA*, GBFS). ENHSP was chosen because all ten domains need numeric fluents and quantified formation actions in the same model, and because it runs as a single Java archive that the simulator can call as a subprocess.

Plans can be checked independently with the VAL validator [12], which replays each action against the model and reports any violated precondition.

2.4 Planning for UAVs and multi-drone formations

Formation control for multi-agent systems is usually solved at the continuous level with leader–follower, virtual-structure or behaviour-based controllers [13, 14]. Surveys of aerial swarm robotics [15] separate this low-level control from higher-level mission planning and task allocation. Symbolic planners have been used at the higher level for UAV missions, for example temporal-logic planning and execution monitoring for single and multiple UAVs [16].

This project follows that split. PDDL decides the discrete mission logic — who is ready, when the formation may move, when a lap or a layer is complete — while Python geometry and the flight controller handle continuous offsets, speeds and separation. Chapter 5 shows how leader–follower ideas appear in the models as a leader action followed by a single close-up action for all followers.

2.5 Research gap addressed

Most formation work assumes a fixed fleet size and a hand-coded sequence of phases. Here, one template per mission is expanded automatically to the selected fleet size, so the planner re-derives the launch order, readiness checks and lap counting for each instance. The same pipeline also serves non-formation missions such as search and road inspection.

Chapter 3 · System overview

The planning layer sits between the operator’s map and the flight controllers: it turns clicked locations and a chosen fleet into a PDDL problem, asks ENHSP for a plan, and hands per-drone routes to the execution backend.

3.1 The simulator

The simulator is a Python/PySide6 desktop application. The operator picks a mission from a *Named Plan* list, ticks the drones to fly, and marks the mission inputs on a map. Each simulated drone runs in its own thread with its own flight-controller model, and talks to

the ground control station over UDP messages and MAVLink-style mission items.

3.2 Planning pipeline

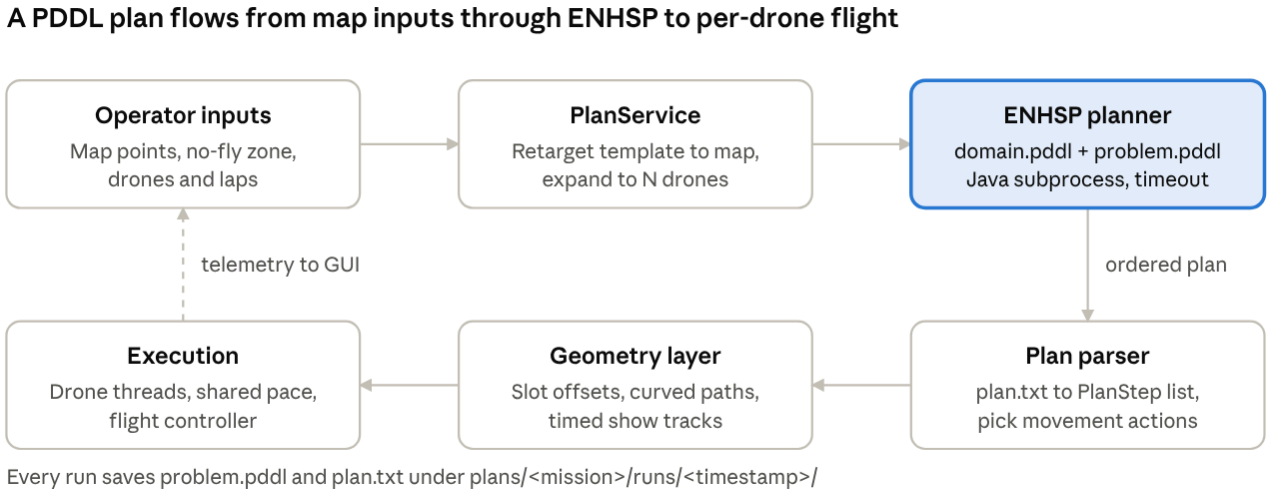


Figure 3.1 · Planning pipeline, 6 stages

Only the ENHSP stage reasons over PDDL; the stages before it write the problem file and the stages after it turn the plan into motion.

Stage	Module	Job
Plan discovery	services/plan_service.py	Lists folders under plans/ with a domain and a problem file; picks the mission kind from the domain name
Retargeting	engine/pddl_problem.py and per-mission modules	Moves the template's coordinates onto the clicked points, recomputes edge distances and energy, adds detours around restricted areas
Fleet expansion	expand_*_drones functions	Clones the reference drone for the selected fleet and rebuilds slot, layer and neighbour facts
Solving	engine/pddl_planner.py	Runs ENHSP as a Java subprocess with a timeout and parses the plan into PlanStep records

Stage	Module	Job
Route extraction	services/plan_service.py	Reads the movement actions of the plan and builds the leader corridor or per-drone routes
Geometry	figure8_problem.py, star_problem.py, planets_problem.py, search_problem.py, road_inspection.py	Adds formation offsets, curved paths, timed show tracks and lawnmower lanes
Execution	services/thread_backend.py, engine/train_keeping.py, engine/mavlink_interpreter.py	Sends routes to each drone and keeps trains and shows in step with a shared pace

3.3 Mission kind dispatch

The mission kind is chosen from the domain name inside domain.pddl, not from the folder name. This lets a new mission be added by dropping a folder into plans/ and registering its domain name.

Folder	Domain name	Kind	Operator inputs
travell	swarm-drone-mission	point_to_point	Source, destination, optional restricted area
Search	forest-drone-search	area_coverage	Base and search polygon
vformation	v-formation-drone-mission	formation	Source, destination, drones
gridformation	grid-formation-drone-mission	grid_formation	Source, destination, drones
3dgridformation	cube-grid-formation-drone-mission	cube_formation	Source, destination, drones
parabolaformation	parabola-formation-drone-mission	parabola_formation	Source, destination
figure8formation	figure8-formation-drone-mission	figure8_formation	Base, left end, crossing, right end, laps
starformation	star-formation-drone-mission	star_formation	Show centre, drones, laps

Folder	Domain name	Kind	Operator inputs
planets	planets-formation-drone-mission	planets_formation	Sun centre, drones, laps
Inspect	road-inspection-drone-mission	road_inspection	Source and destination on a road

3.4 Division of work between PDDL and geometry

PDDL plans the discrete logic: which drone may take off, when the formation is established, which leg comes next, how many laps remain. Python supplies what symbolic waypoints cannot express: metre-level slot offsets, curve shapes, timing and separation checks. In formation models the leader and members share a symbolic location after a close-up action; this means they share a formation reference point, and their real positions are separated by along-track, cross-track and altitude offsets.

Chapter 4 · PDDL modelling methodology

All ten domains share one design pattern: typed drones and locations, a common set of health predicates, numeric fluents for battery and altitude, a launch phase, a mission-specific core, and a shared set of failure-detection actions.

4.1 Domain statistics

Domain	Types	Predicates	Functions	Actions
Figure-eight	drone, location, controller	33	20	17
3-D cube	drone, location, controller, layer	32	20	15
Parabola	drone, location, controller	31	20	15
Travel	drone, location, waypoint, controller	30	11	16
V formation	drone, location, controller, direction	30	18	14
Search	drone, location, forest-area, controller	29	15	13
2-D grid	drone, location, controller	28	18	14
Star show	drone, location, controller	27	14	16
Planets	drone, location, controller	24	14	11

Domain	Types	Predicates	Functions	Actions
Road inspection	drone, location, controller	20	15	11
Total		**284**	**165**	**142**

Travel and Search declare five requirements (`:strips :typing :numeric-fluents :negative-preconditions :conditional-effects`). The eight newer domains add `:equality :disjunctive-preconditions :universal-preconditions` .

4.2 State representation

The state of each drone is split into three groups.

- **Position and phase (predicates):** (at ?d ?l), (flying ?d), (at-cruise-altitude ?d), (in-formation ?d), (landed ?d), (mission-completed ?d).
- **Health (predicates):** (gps-ok ?d), (communication-ok ?d), (drone-healthy ?d), (neighbor-comm-ok ?d). Failure flags such as (gps-lost ?d) are set by detection actions.
- **Resources (numeric fluents):** (battery ?d), (altitude ?d), (target-altitude ?d), (energy-required ?from ?to), (climb-energy ?d), (cruise-leg-energy ?d), (health ?d), and mission counters such as (laps-completed) or (launch-delay).

Battery values are model units from 0 to 100. They express relative cost and feasibility, not calibrated watt-hours.

4.3 Movement with numeric resources

The basic movement action in the travel domain shows the pattern used everywhere: the move is allowed only if the drone is healthy, a safe route has been planned and the battery strictly exceeds the edge cost.

```
(:action travel
  :parameters (?d - drone ?from - location ?to - location)
  :precondition (and (at ?d ?from) (route-planned ?d) (safe-route ?from ?to)
                     (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                     (> (battery ?d) (energy-required ?from ?to)))
  :effect (and (at ?d ?to) (not (at ?d ?from)) (flying ?d)
               (decrease (battery ?d) (energy-required ?from ?to))
               (not (route-planned ?d))))
```

The effect deletes route-planned, so the planner must re-run the safety check before the next leg. This *check–act–invalidate* idiom forces every leg to pass the safety gate again.

4.4 Whole-formation actions

A formation must move as one unit. Instead of one action per follower, the models use a universal precondition to require that every member is ready, and conditional effects to update every member at once. In the V formation:

```
(:action close-up-wings
  :parameters (?lead - drone ?to - location)
  :precondition (and (v-formation-established) (not (wings-closed))
     (formation-leader ?lead) (at ?lead ?to) (formation-at ?to)
     (forall (?w - drone)
       (or (not (wing-drone ?w))
           (and (in-formation ?w) (gps-ok ?w) (communication-ok ?w)
                (drone-healthy ?w) (neighbor-comm-ok ?w)
                (> (battery ?w) (cruise-leg-energy ?w))))))
  :effect (and
     (forall (?w - drone) (forall (?x - location)
       (when (and (wing-drone ?w) (at ?w ?x)) (not (at ?w ?x)))))
     (forall (?w - drone) (when (wing-drone ?w) (at ?w ?to)))
     (forall (?w - drone) (when (wing-drone ?w)
       (decrease (battery ?w) (cruise-leg-energy ?w))))
     (wings-closed)))
```

This gives an all-or-nothing move: if any member is unhealthy or short of battery, no member moves. The plan length also grows linearly with the number of legs, not with legs × drones.

4.5 Leader-gate alternation

formation-cruise moves the leader one leg and deletes (wings-closed) (or train-closed, grid-closed). The leader cannot start another leg until the close-up action restores that flag. The plan therefore alternates leader leg and close-up, so the formation never stretches by more than one leg.

4.6 Launch ordering

Take-off is ordered to avoid conflicts near the ground. In the V and grid domains the leader takes off first and uses hold-position, which increases (launch-delay); members may take off only once that counter reaches their delay. In the 3-D cube a layer gate releases one layer at a time:

```
(:action open-next-layer
  :parameters (?lo - layer ?hi - layer)
  :precondition (and (layer-below ?lo ?hi) (layer-launched ?lo)
                     (not (layer-launched ?hi))
                     (forall (?d - drone)
                       (or (not (in-layer ?d ?lo)) (at-cruise-altitude ?d))))
  :effect (and (layer-launched ?hi)
               (forall (?d - drone)
                 (when (in-layer ?d ?hi) (cleared-to-launch ?d)))))
```

4.7 Counting laps with numeric fluents

Show missions repeat a pattern. The number of laps is a numeric fluent set by the operator, so the same domain plans 1 or 10 laps. In the figure-eight domain start-next-lap is applicable only while more laps remain, and finish-laps counts the final one:

```
(:precondition (and ... (< (+ (laps-completed) 1) (laps-required))))
(:effect (and ... (increase (laps-completed) 1)))
```

4.8 Failure detection

Every domain includes detect-gps-loss, detect-communication-loss and detect-health-failure; formation domains add detect-neighbor-link-loss, and most add notify-ground-controller. These actions set failure flags and remove (flying) or (in-formation), which makes the normal goal unreachable. They are not used in a nominal plan; they define how the state must change when monitoring reports a fault, so that replanning starts from a correct state.

4.9 Problem file structure

Each problem file is a template with a reference fleet (often a leader and one or two members), real coordinates as numeric fluents, slot offsets and the goal. The simulator rewrites it for each run. The V-formation template places each wing slot 17.32 m behind and 10 m beside the apex, a 60° V, and its goal is:

```
(:goal (and (v-formation-established)
            (in-formation drone-lead) (in-formation drone-left) (in-formation
drone-right)
            (at-cruise-altitude drone-lead) (at-cruise-altitude drone-left)
            (at-cruise-altitude drone-right)
            (>= (altitude drone-lead) 50) (>= (altitude drone-left) 50)
            (>= (altitude drone-right) 50)
            (formation-at destination)
            (at drone-lead destination) (at drone-left destination)
            (at drone-right destination)
            (formation-mission-complete)
            (mission-completed drone-lead) (mission-completed drone-left)
            (mission-completed drone-right)))
```

Chapter 5 · Mission and formation models

Each of the ten missions has its own domain and problem file under plans/. This chapter gives, for each, the purpose, the nominal action sequence, the goal and the special modelling idea it introduces. Full sources are listed in Appendix A.

5.1 Point-to-point travel (plans/travell)

The swarm flies one shared route from source to destination, optionally around a restricted area. The template's single reference drone is cloned for the selected fleet.

- **Nominal sequence:** check-distance → check-battery → plan-safe-route → travel (repeated per leg) → reach-destination.
- **Goal:** (at drone1 destination) and (mission-completed drone1).
- **Template values:** battery 85, minimum-return-battery 30, health 95.
- **Fault actions (8):** GPS, communication and health detection; notify-ground-controller; controller-order-return, return-to-source, controller-order-hover, hover, initiate-low-battery-return, sacrifice-drone.
- **Idea:** check–act–invalidate. travel deletes route-planned, so every leg needs a fresh safety check.

5.2 Forest search (plans/Search)

Each drone sweeps its own lane of a polygon marked by the operator. engine/search_problem.py splits the polygon into equal-area lawnmower lanes and inserts a coarse skeleton of each lane into the PDDL problem. At least two drones are required.

- **Nominal sequence (per drone):** check-battery → check-drone-separation → move-search → start-search → cover-area → … → complete-forest-search → return-to-base → complete-drone-mission.
- **Goal:** (forest-search-completed), every drone's mission completed and every drone back at base. For three or more drones the generator adds (area-covered forest-laneN) for every lane.
- **Template values:** 10 m minimum separation between drones, 50 m coverage width, battery 100.
- **Idea:** move-search deletes battery-checked and separation-ok, so separation between lanes is re-checked before every move.

5.3 V formation (plans/vformation)

A leader holds the apex and r ranks extend along each wing, so N = 1 + 2r (odd, at least 3). Each rank sits 17.32 m behind and 10 m beside the one ahead.

- **Nominal sequence:** pre-flight-check → takeoff → climb-to-cruise-altitude for the leader; hold-position until the wing launch delay (5) is reached; then each wing drone takes off and climbs; take-formation-slot for all; establish-v-formation; then formation-cruise and close-up-wings alternate per leg; complete-formation-mission.
- **Goal:** V established, all drones in formation at 50 m or above, at the destination, missions completed.
- **Topology:** (wing-neighbor) chains from the leader out along each wing. The leader has 2 neighbours, inner wing drones 2, wing tips 1.

5.4 Two-dimensional grid (plans/gridformation)

Drones fly a horizontal R × C grid with 10 m spacing; 4 drones give 2 × 2, 9 give 3 × 3, and other counts use a rectangular factorisation.

- **Nominal sequence:** launch phase as in the V → take-grid-slot → establish-grid-formation → (formation-cruise, close-up-grid) per leg → complete-grid-mission.
- **Topology:** 4-neighbourhood (front, back, left, right). Corners have 2 neighbours, edges 3, interior drones 4. A full grid has R(C − 1) + C(R − 1) neighbour pairs.

5.5 Three-dimensional cube (plans/3dgridformation)

The grid is stacked into layers; 8 drones form a 2 × 2 × 2 cube. Each layer flies at its own altitude.

- **New objects and facts:** layer, (in-layer ?d ?l), (layer-below ?lo ?hi), (layer-launched ?l), (cleared-to-launch ?d).

- **Nominal sequence:** bottom layer takes off and climbs → open-next-layer (only when every drone of the lower layer is at cruise altitude) → next layer takes off → … → take-cube-slot → establish-cube-formation → (formation-cruise, close-up-cube) per leg → complete-cube-mission.
- **Idea:** only one layer is ever climbing, so no drone ascends through another's slot.
- **Topology:** up to 24 neighbours — 8 in the same layer, 8 above and 8 below, excluding the cells directly overhead and underneath.

5.6 Parabola (plans/parabolaformation)

Exactly three drones fly abreast: a vertex at 50 m and two arms 10 m to each side at 60 m, giving an upward-opening parabola seen across the track.

- **Nominal sequence:** pre-flight-check, takeoff and climb for all three → establish-comm-link for each ordered pair → confirm-swarm-comm for each drone → take-parabola-slot → establish-parabola-formation → (formation-cruise, hold-parabola-shape) per leg → complete-parabola-mission.
- **Idea:** the only domain with explicit drone-to-drone links. (link-up ?a ?b) is created by an action and checked before the shape may be held; detect-neighbor-link-loss drops every link of the failed drone. establish-parabola-formation also checks that each arm is exactly arm-rise above the vertex and that no two drones are closer than the minimum separation.
- **Topology:** a complete graph of 3 drones, 3 pairs, 6 directed links.

5.7 Figure-eight train (plans/figure8formation)

The operator marks the base, the left lobe end, the middle crossing and the right lobe end, and sets the lap count. Drones fly in ordered slots along one shared curved path.

- **Nominal sequence:** pre-flight-check → takeoff → climb → take-train-slot → establish-figure8-formation → fly-to-pattern → (formation-cruise, follow-leader) along the 8 → start-next-lap while laps remain → finish-laps → return-to-base → follow-leader → complete-figure8-mission.
- **Idea:** laps are a numeric counter, and establish-figure8-formation requires that the spacing never brings two drones to the crossing together. The geometry uses a path gap of $10\sqrt{2} \approx 14.14$ m at a right-angle crossing, which gives 10 m straight-line clearance.
- **Topology:** each drone links to the drone ahead and behind in the train (up to 2 peers).

5.8 Star and circle show (plans/starformation)

Drones take off from pads around a centre, form a circle, and in each lap perform six shape steps before landing back on their pads.

- **Lap sequence:** form-star → form-circle-in-lap → spin-circle → form-star-again → spin-star → close-lap.
- **Full sequence:** pre-flight-check → takeoff → climb → form-circle → laps → land-show.
- **Idea:** the domain records the current shape and phase, so the planner cannot skip a step; engine/star_problem.py turns the steps into synchronised timed tracks.
- **Topology:** a ring; each drone links to the previous and next drone (2 peers when N ⩾ 3).

5.9 Planetary orbits (plans/planets)

Two to eight drones play planets around a chosen sun. Orbit k has radius 50(k + 1) m, from 50 m to 400 m, and adjacent orbits turn in opposite directions.

- **Nominal sequence:** pre-flight-check → takeoff → climb → enter-orbits → fly-lap (per lap) → land-planets.
- **Idea:** enter-orbits is allowed only when every drone is airborne and no two neighbouring orbits turn the same way. One lap is one revolution of the outermost drone.
- **Topology:** an open chain by orbit index; innermost and outermost drones have 1 neighbour, the rest 2.

5.10 Road inspection (plans/Inspect)

One drone follows a real road between two points, inspects each segment, flies back and lands. The road shape comes from an OSRM routing service, and its segments replace the template's single reference segment.

- **Nominal sequence:** pre-flight-check → takeoff → climb → inspect-segment (per segment) → finish-inspection → return-along-road (per segment) → land-at-source.
- **Goal:** (road-inspected), drone at source, landed, mission completed.

5.11 Communication topology summary

Mission	Logical neighbours	Max peers per drone	Links for N drones
Travel	None	0	0
Search	Pairwise separation between adjacent lanes	Not a radio graph	—
V formation	Chain along each wing	2 (tips 1)	N − 1
2-D grid	Front, back, left, right	4	R(C − 1) + C(R − 1)

Mission	Logical neighbours	Max peers per drone	Links for N drones
3-D cube	Same layer, above, below	24	Depends on filled cells
Parabola	Every other drone	2	3
Figure-eight	Ahead and behind in the train	2	N − 1 or N
Star show	Previous and next on the ring	2	N
Planets	Adjacent orbits	2 (ends 1)	N − 1
Road inspection	Single drone	0	0

These relations are planning facts. Except in the parabola, actions check the per-drone flag (neighbor-comm-ok ?d) rather than each pair; runtime messages are carried by the GCS link described in Chapter 6.

Chapter 6 · Plan generation, solving and execution

A planning run takes about one second: the template is rewritten for the map and fleet, saved with a timestamp, solved by ENHSP in a background thread, and its movement actions are turned into flight routes.

6.1 Problem generation

1. **Read the template** — problem.pddl (SearchProblem.pddl for search) from the mission folder.
2. **Retarget** — replace the template's latitude/longitude fluents with the clicked points. The reference route is rotated, scaled and translated; edge distances and (energy-required ?a ?b) are recomputed. If a restricted area blocks the route, buffered detour waypoints are inserted; formation missions widen the buffer by the formation's lateral reach.
3. **Expand the fleet** — clone the reference drones and rebuild slot offsets, layer membership, launch delays and neighbour facts for the chosen N. For example, expand_formation_drones chains wing-neighbor rank to rank instead of copying the rank-1 pair.
4. **Save** — write the result to plans/<mission>/runs/<timestamp>/problem.pddl, so every run can be reproduced.

6.2 Solving with ENHSP

engine/pddl_planner.py locates Java and the ENHSP jar (overridable with the ENHSP_JAR variable) and runs:

```
java -jar tools/enhsp/enhsp-dist/enhsp.jar -o <domain.pddl> -f <problem.pddl>
[options]
```

Mission	Planner options	Timeout
Travel, V formation, parabola	ENHSP default	30 s
2-D grid	-gro naive	max(30, R·C) s
3-D cube	-gro naive	max(45, 1.5·L·R·C) s
Figure-eight	-gro naive	max(30, N·laps) s
Star, planets	-gro naive	max(30, N + 2·laps) s
Road inspection	ENHSP default	60 s
Search	-s gbfs -h hadd -ha true	max(45, 15·N) s

Naive grounding (-gro naive) is used for domains with nested universal quantifiers. Search uses greedy best-first search with the h-add heuristic and helpful actions: with several separation-coupled lanes the default WA* did not return in time, while GBFS solves in seconds, and optimality is not needed because drones fly the generated lanes.

The planner runs on a Qt worker thread, so the GUI stays responsive; it reports back through finished and failed signals. The console output is saved as plan.txt beside the problem.

6.3 Parsing the plan

parse_plan reads lines of the form `3.0: (travel drone1 source waypoint1)` into PlanStep(index, action, args) records and ignores ENHSP's log lines. If ENHSP runs but no step is parsed, PlanNotFound is raised; a missing Java, missing jar or timeout raises PlannerError. The GUI shows these as different failure messages.

6.4 From actions to routes

Mission	Actions read	Output
Travel	travel	One shared route, flown by every selected drone

Mission	Actions read	Output
Search	move-search, return-to-base	Checks every drone moves; drones fly their fine lawnmower lanes
V, grid, cube, parabola	formation-cruise	Leader corridor; members get parallel routes at their slot offsets (cube adds per-layer altitude)
Figure-eight	Lap actions and lap count	Approach, loop × laps, departure for every drone (TrainRoute)
Star, planets	Shape and lap actions	Synchronised timed tracks per drone
Road inspection	inspect-segment, return-along-road	Out-and-back route along the road

6.5 Execution

Routes are sent to each drone thread by services/thread_backend.py. Commands (NAVIGATE, ARM, TAKEOFF, HOLD, RESUME, RETURN_TO_LAUNCH, LAND, TRAIN_PACE) travel as JSON over UDP from the GCS on port 14690 to each drone on port 14700 + SYSID; drones return telemetry, heartbeats and acknowledgements. For trains and shows, the GCS broadcasts a shared pace and engine/train_keeping.py lets each drone correct its own speed from its progress along the route. The MAVLink path (engine/mavlink_mission.py, engine/mavlink_interpreter.py) turns a solved route into takeoff, waypoint and landing mission items.

6.6 Example: solved V-formation plan

The stored 3-drone V template solves in 21 steps:

```
 0 pre-flight-check drone-lead source source
 1 takeoff drone-lead source
 2 climb-to-cruise-altitude drone-lead drone-left
 3-7 hold-position drone-lead (x5)
 8 pre-flight-check drone-right source left
 9 takeoff drone-right source right
10 climb-to-cruise-altitude drone-right lead source
                                                  source destination
                                                  destination
                                                  drone-lead destination
11 pre-flight-check drone-left
12 takeoff drone-left source
13 climb-to-cruise-altitude
14 take-formation-slot drone-lead
15 take-formation-slot drone-left
16 take-formation-slot drone-right
17 establish-v-formation drone-
18 formation-cruise drone-lead
19 close-up-wings drone-lead
20 complete-formation-mission
```

The five hold-position steps are the wing launch delay: wing drones cannot take off until the leader has held for 5 counter units. One close-up-wings action moves both wing drones.

Chapter 7 · Results and discussion

All 15 generated mission instances were solved, in 0.4 to 2.4 s end to end, and plan length grows by exactly 4 steps per added drone in the formation missions. Nine of the ten stored templates also solve directly.

7.1 Experimental setup

- ENHSP (tools/enhsp/enhsp-dist/enhsp.jar) on OpenJDK 17.0.20, Windows 11, run on 23 October 2026.
- **Templates:** each stored domain–problem pair solved with default ENHSP settings.
- **Generated instances:** produced by the project's own PlanService code (retarget, expand, solve, extract) with planner options as in Section 6.2, using map points near Chennai (source 12.9916 N, 80.2336 E; destination about 1.3 km north-east). Runs were written to a scratch copy of plans/, so the repository was not changed.
- Road inspection was not included in the generated set because it needs the online OSRM routing service.

7.2 Stored templates

Template	Solved	Plan steps	Expanded nodes	Planning time (ms)
Figure-eight	Yes	31	32	38
Parabola	Yes	25	26	35
V formation	Yes	21	22	25
3-D cube	Yes	17	18	24
2-D grid	Yes	17	18	18
Travel	Yes	9	10	17
Road inspection	Yes	7	8	16
Star show	Yes	14	15	15
Planets	Yes	9	10	6
Search	No (seed only)	—	—	—

The search template contains only the base location and lane assignments; it has no waypoints or connected edges, so ENHSP's AIBR pre-processing proves it unsolvable before search. This is by design: engine/search_problem.py inserts the lane waypoints at run time, and every generated search instance below is solvable.

7.3 Generated mission instances

Mission	Drones	Plan steps	Ground actions	Expanded nodes	Planning time (ms)	End-to-end (s)
Figure-eight, 2 laps	4	73	49	137	95	2.29
Search	4	61	612	110	304	1.30
Figure-eight, 1 lap	4	56	49	57	55	2.38
2-D grid (3 × 3)	9	45	44	46	102	0.74
3-D cube (2 × 2 × 2)	8	42	41	43	76	0.66
V formation	7	37	36	38	108	1.21
Search	2	35	150	42	69	0.67
V formation	5	29	28	30	63	0.63
Star show, 1 lap	6	26	26	27	33	0.48
2-D grid (2 × 2)	4	25	24	26	38	0.57
Parabola	3	25	28	26	35	0.53

Mission	Drones	Plan steps	Ground actions	Expanded nodes	Planning time (ms)	End-to-end (s)
V formation	3	21	20	22	35	0.60
Planets, 1 lap	4	15	15	16	21	0.60
Travel with restricted area	1	7	30	8	16	0.43
Travel	1	5	12	6	11	0.54

Planning time is the “Planning Time” ENHSP reports; end-to-end time also includes problem generation, JVM start-up, grounding and route extraction.

7.4 Observations

1. **Linear scaling with fleet size.** The V formation grows from 21 to 29 to 37 steps for 3, 5 and 7 drones, and the grid from 25 to 45 steps for 4 and 9 drones: 4 steps per added drone (pre-flight-check, takeoff, climb, take-slot). The formation-level close-up actions keep movement cost independent of N.
2. **Near-greedy search.** In most formation instances expanded nodes = plan steps + 1, so the heuristic leads ENHSP almost directly to the goal. The models’ gates (route-planned, wings-closed, launch-delay) remove most wrong choices.
3. **Search is the hardest domain.** Separation coupling between lanes makes grounding grow fast: 150 ground actions for 2 drones and 612 for 4. This is why GBFS with h-add and helpful actions is used for search.
4. **Laps cost little.** A second figure-eight lap adds 17 steps and 40 ms, since the lap actions are reused with a numeric counter.
5. **Restricted areas.** Adding a no-fly zone to the travel mission raised the ground actions from 12 to 30 and the plan from 5 to 7 steps: one extra plan-safe-route and travel pair for the detour leg.

7.5 Historical usage

The simulator’s own logs (plans/PLANS_STATUS.txt and saved runs) record runs made through the GUI during development.

Mission	Runs logged	Solved	Note
Travel	83	76	Failures were infeasible route and restricted-area choices in early testing
Search	69	69	No failures

Mission	Runs logged	Solved	Note
V formation	65	62	2 infeasible inputs, 1 run cut short when the app closed
2-D grid	19	15	All 4 failures before a fix on 2026-09-16
Figure-eight	15	14	Saved runs in plans/figure8formation/runs
Parabola	9	9	Saved runs

These are development logs, not a controlled benchmark.

7.6 Model review notes

- In the stored travel template plan, step 4 is plan-safe-route drone1 waypoint1 source, a check for a leg that is never flown. It does no harm because the next travel re-plans, but an extra precondition tying the check to the next leg would shorten plans.
- The travel domain uses or in some conditions without declaring :disjunctive-preconditions, and the search domain uses = without :equality. ENHSP accepts both, but stricter planners may not.
- Battery checks compare against the next action's cost only. minimum-return-battery is not enforced on every move, so a full return reserve is not guaranteed by the model.

Chapter 8 · Conclusion and future work

One PDDL planning pipeline now drives ten different swarm missions, from single-drone road inspection to an eight-drone cube, and every generated test instance was solved in under 2.5 s.

8.1 Conclusions

- Ten missions are modelled as PDDL 2.1 domains with 142 actions, using numeric fluents for battery, altitude and lap counts.
- Universal preconditions and conditional effects let one action check and move a whole formation, so plan length grows by only 4 steps per added drone.
- Gating predicates (route-planned, wings-closed, layer-launched, laps-required) encode safety and ordering rules directly in the model: re-checking before every leg, one layer climbing at a time, no crossing conflicts at the figure-eight centre.
- Template retargeting and fleet expansion make one problem file serve any map location and any valid fleet size.

- The division of work is clear: PDDL decides discrete mission logic; Python geometry and the flight controller handle continuous motion.

8.2 Limitations

- Actions are instantaneous; timing, acceleration and continuous collision avoidance are outside the PDDL model.
- Battery and energy are model units and are not calibrated against real flight endurance.
- Failure-detection actions exist, but live telemetry does not yet update the planning state, so there is no automatic replanning.
- Most neighbour links are recorded as facts but checked through one per-drone flag; only the parabola checks each link.
- Results come from simulation; no real-flight validation was performed.

8.3 Future work

1. **Execution monitoring and replanning:** map telemetry faults onto failure predicates and replan from the current state with recovery goals (return, hover, land).
2. **Temporal planning:** use PDDL 2.1 durative actions so the planner reasons about flight time and overlapping actions.
3. **Return-energy reserve:** require (battery − cost) ⩾ return cost on every move.
4. **Per-link checks:** extend the parabola's explicit link model to grid, cube and train formations.
5. **Formal validation:** check every generated plan with VAL and fix the missing requirement declarations.
6. **Formation transitions:** plan changes between shapes (V to grid, grid to cube) within one mission.
7. **Calibration and hardware tests:** fit the energy model to SITL logs and fly selected missions on real drones.

References

1. R. E. Fikes and N. J. Nilsson, "STRIPS: A new approach to the application of theorem proving to problem solving," *Artificial Intelligence*, vol. 2, no. 3–4, pp. 189–208, 1971.
2. M. Ghallab, D. Nau and P. Traverso, *Automated Planning: Theory and Practice*. Morgan Kaufmann, 2004.
3. B. Bonet and H. Geffner, "Planning as heuristic search," *Artificial Intelligence*, vol. 129, no. 1–2, pp. 5–33, 2001.

4. J. Hoffmann and B. Nebel, “The FF planning system: Fast plan generation through heuristic search,” *Journal of Artificial Intelligence Research*, vol. 14, pp. 253–302, 2001.
5. D. McDermott et al., “PDDL — The Planning Domain Definition Language,” Yale Center for Computational Vision and Control, Tech. Rep. CVC TR-98-003, 1998.
6. M. Fox and D. Long, “PDDL2.1: An extension to PDDL for expressing temporal planning domains,” *Journal of Artificial Intelligence Research*, vol. 20, pp. 61–124, 2003.
7. S. Edelkamp and J. Hoffmann, “PDDL2.2: The language for the classical part of the 4th International Planning Competition,” Univ. of Freiburg, Tech. Rep. 195, 2004.
8. A. Gerevini and D. Long, “Plan constraints and preferences in PDDL3,” Univ. of Brescia, Tech. Rep., 2005.
9. J. Hoffmann, “The Metric-FF planning system: Translating ‘ignoring delete lists’ to numeric state variables,” *Journal of Artificial Intelligence Research*, vol. 20, pp. 291–341, 2003.
10. E. Scala, P. Haslum, S. Thiébaux and M. Ramirez, “Interval-based relaxation for general numeric planning,” in *Proc. ECAI*, 2016, pp. 655–663.
11. E. Scala, P. Haslum, S. Thiébaux and M. Ramirez, “Subgoaling techniques for satisficing and optimal numeric planning,” *Journal of Artificial Intelligence Research*, vol. 68, pp. 691–752, 2020.
12. R. Howey, D. Long and M. Fox, “VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL,” in *Proc. IEEE ICTAI*, 2004, pp. 294–301.
13. K.-K. Oh, M.-C. Park and H.-S. Ahn, “A survey of multi-agent formation control,” *Automatica*, vol. 53, pp. 424–440, 2015.
14. R. W. Beard, J. Lawton and F. Y. Hadaegh, “A coordination architecture for spacecraft formation control,” *IEEE Trans. Control Systems Technology*, vol. 9, no. 6, pp. 777–790, 2001.
15. S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen and V. Kumar, “A survey on aerial swarm robotics,” *IEEE Trans. Robotics*, vol. 34, no. 4, pp. 837–855, 2018.
16. P. Doherty, J. Kvarnström and F. Heintz, “A temporal logic-based planning and execution monitoring framework for unmanned aircraft systems,” *Autonomous Agents and Multi-Agent Systems*, vol. 19, no. 3, pp. 332–377, 2009.
17. Project sources: plans/ (ten domain–problem pairs), services/plan_service.py, engine/pddl_planner.py, engine/pddl_problem.py, engine/search_problem.py, engine/figure8_problem.py, engine/star_problem.py, engine/planets_problem.py, engine/road_inspection.py, plans/PLANS_STATUS.txt.
18. docs/PDDL_Planning_Report.pdf — earlier planning-component report, including full verbatim PDDL listings.

Bibliographic details were compiled without online lookup; check volume and page numbers against the publisher before printing.

Appendix A · Reproducing the results

Install the dependencies in requirements.txt, Java 17 or later, and the ENHSP jar in tools/enhsp/enhsp-dist/. Solve a stored template directly:

```
java -jar tools/enhsp/enhsp-dist/enhsp.jar -o plans/vformation/domain.pddl -f plans/vformation/problem.pddl
```

To reproduce a GUI run, run python main.py, choose the mission and drones, mark the inputs, and plan. Use the problem.pddl and plan.txt saved under plans/<mission>/runs/<timestamp>/; the template alone does not hold the chosen coordinates or fleet size.

Appendix B · Action catalogue

Domain	Actions
Travel (16)	check-distance, check-battery, notify-insufficient-battery, plan-safe-route, travel, reach-destination, detect-gps-loss, detect-communication-loss, detect-health-failure, notify-ground-controller, controller-order-return, return-to-source, controller-order-hover, hover, initiate-low-battery-return, sacrifice-drone
Search (13)	check-battery, notify-insufficient-battery, check-drone-separation, start-search, cover-area, move-search, complete-forest-search, return-to-base, complete-drone-mission, emergency-low-battery-return, detect-gps-loss, detect-communication-loss, detect-health-failure
V formation (14)	pre-flight-check, takeoff, climb-to-cruise-altitude, hold-position, take-formation-slot, establish-v-formation, formation-cruise, close-up-wings, complete-formation-mission, detect-gps-loss, detect-communication-loss, detect-neighbor-link-loss, detect-health-failure, notify-ground-controller

Domain	Actions
2-D grid (14)	pre-flight-check, takeoff, climb-to-cruise-altitude, hold-position, take-grid-slot, establish-grid-formation, formation-cruise, close-up-grid, complete-grid-mission, detect-gps-loss, detect-communication-loss, detect-neighbor-link-loss, detect-health-failure, notify-ground-controller
3-D cube (15)	pre-flight-check, takeoff, climb-to-cruise-altitude, hold-position, open-next-layer, take-cube-slot, establish-cube-formation, formation-cruise, close-up-cube, complete-cube-mission, detect-gps-loss, detect-communication-loss, detect-neighbor-link-loss, detect-health-failure, notify-ground-controller
Parabola (15)	pre-flight-check, takeoff, climb-to-cruise-altitude, establish-comm-link, confirm-swarm-comm, take-parabola-slot, establish-parabola-formation, formation-cruise, hold-parabola-shape, complete-parabola-mission, detect-gps-loss, detect-communication-loss, detect-neighbor-link-loss, detect-health-failure, notify-ground-controller
Figure-eight (17)	pre-flight-check, takeoff, climb-to-cruise-altitude, take-train-slot, establish-figure8-formation, fly-to-pattern, formation-cruise, follow-leader, start-next-lap, finish-laps, return-to-base, complete-figure8-mission, detect-gps-loss, detect-communication-loss, detect-neighbor-link-loss, detect-health-failure, notify-ground-controller
Star show (16)	pre-flight-check, takeoff, climb-to-cruise-altitude, form-circle, form-star, form-circle-in-lap, spin-circle, form-star-again, spin-star, close-lap, land-show, detect-gps-loss, detect-communication-loss, detect-neighbor-link-loss, detect-health-failure, notify-ground-controller
Planets (11)	pre-flight-check, takeoff, climb-to-cruise-altitude, enter-orbits, fly-lap, land-planets, detect-gps-loss, detect-communication-loss, detect-neighbor-link-loss, detect-health-failure, notify-ground-controller

Domain	Actions
Road inspection (11)	pre-flight-check, takeoff, climb-to-cruise-altitude, inspect-segment, finish-inspection, return-along-road, land-at-source, detect-gps-loss, detect-communication-loss, detect-health-failure, notify-ground-controller

The complete domain and problem sources are in plans/ and are reproduced verbatim in Appendix A of docs/PDDL_Planning_Report.pdf [18].

Appendix C · Viva preparation

Likely question	Short answer
Why PDDL instead of hand-coded mission logic?	Rules are declared once; the planner finds the order for any fleet size and goal, and each step is traceable to a precondition.
Why ENHSP?	It handles numeric fluents together with forall and when in one model, and runs as a single jar called from Python.
Domain vs problem file?	Domain = types, predicates, functions, actions (per mission type). Problem = objects, initial state, goal (per run).
How does the formation move together?	formation-cruise moves the leader and clears wings-closed; one close-up action with forall/when moves every member and restores it.
How does plan length scale?	Linearly: 4 steps per added drone (pre-flight, takeoff, climb, slot); movement steps do not depend on N.
How is collision avoided?	Symbolically by launch delay, layer gate and crossing-spacing checks; geometrically by slot offsets and clearance checks in Python. Continuous avoidance is not proven by PDDL.
Why is the search template unsolvable?	It is a seed without waypoints; lane waypoints are generated at run time.

Likely question	Short answer
Why GBFS for search?	Separation-coupled lanes blow up WA*; GBFS + h-add + helpful actions solves in seconds and optimality is not needed.
What happens on a failure?	Detection actions define the state change; live replanning from telemetry is future work.
What are the main limitations?	Instantaneous actions, uncalibrated energy, no live replanning, aggregate neighbour flag, simulation only.